🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 13: Identifying and Debugging Stack Buffer Overflows Using Address Sanitizer
```

Scenario

Address Sanitizer (ASan) is a fast memory error detector integrated into modern GCC and Clang compilers. It uncovers out-of-bounds accesses (stack/heap/global buffer overflows), use-after-free, and initialization order bugs that traditional debuggers (or even Valgrind) miss when errors do not immediately cause a Segmentation Fault.

Key Address Sanitizer mechanisms:
1. **Off-by-One Detection**: In classic array indexing logic (`int a[2] = {1, 0}; int b = a[2];`), accessing index 2 reads out-of-bounds memory on the stack frame. Without sanitizers, this undefined behavior silent reads adjacent stack data without crashing.
2. **Compiler Instrumentation**: Adding `-fsanitize=address -ggdb` flags embeds shadow memory checks around every stack allocation and pointer dereference.
3. **Build System Integration**: When compiling source packages via GNU autotools:
`./configure --disable-shared CFLAGS="-fsanitize=address -ggdb" CXXFLAGS="-fsanitize=address -ggdb" LDFLAGS="-fsanitize=address"`
4. **Automated Fuzzing & Crash Enforcement**: By default, ASan logs violations without triggering SIGSEGV. To force immediate process termination during automated fuzzing runs, setting environment variable **export ASAN_OPTIONS='abort_on_error=1'** forces instant aborts on memory violations.

In this exercise, we log into **Ubuntu 20.04 Fuzzing** VM (**student** / **studentpassword**), write out-of-bounds C program **test.c**, compile with Address Sanitizer flags (`gcc -fsanitize=address -ggdb -o test test.c`), execute the binary, and inspect ASan's detailed `stack-buffer-overflow` diagnostic output.

Objective

The objective of this lab is to help students learn how to:
- Authenticate to Ubuntu 20.04 target VM (**student** / **studentpassword**)
- Write C source code exhibiting an off-by-one stack array access error (**test.c**)
- Compile binary with Address Sanitizer and GDB debug symbols: **gcc -fsanitize=address -ggdb -o test test.c**
- Execute compiled test binary (**./test**) and analyze ASan error logs
- Identify stack buffer overflow reports referencing exact source code line numbers
- Configure GNU Autotools build flags for ASan testing
- Enforce abort on error for automated fuzzing integration: **export ASAN_OPTIONS='abort_on_error=1'**

Task 1: Write Out-of-Bounds C Program, Compile with ASan, and Analyze Stack Overflow Logs

In this task, we write an off-by-one array access C program (`test.c`), compile with `-fsanitize=address -ggdb` , execute `./test` , and inspect detailed Address Sanitizer diagnostics.

Lab Tasks

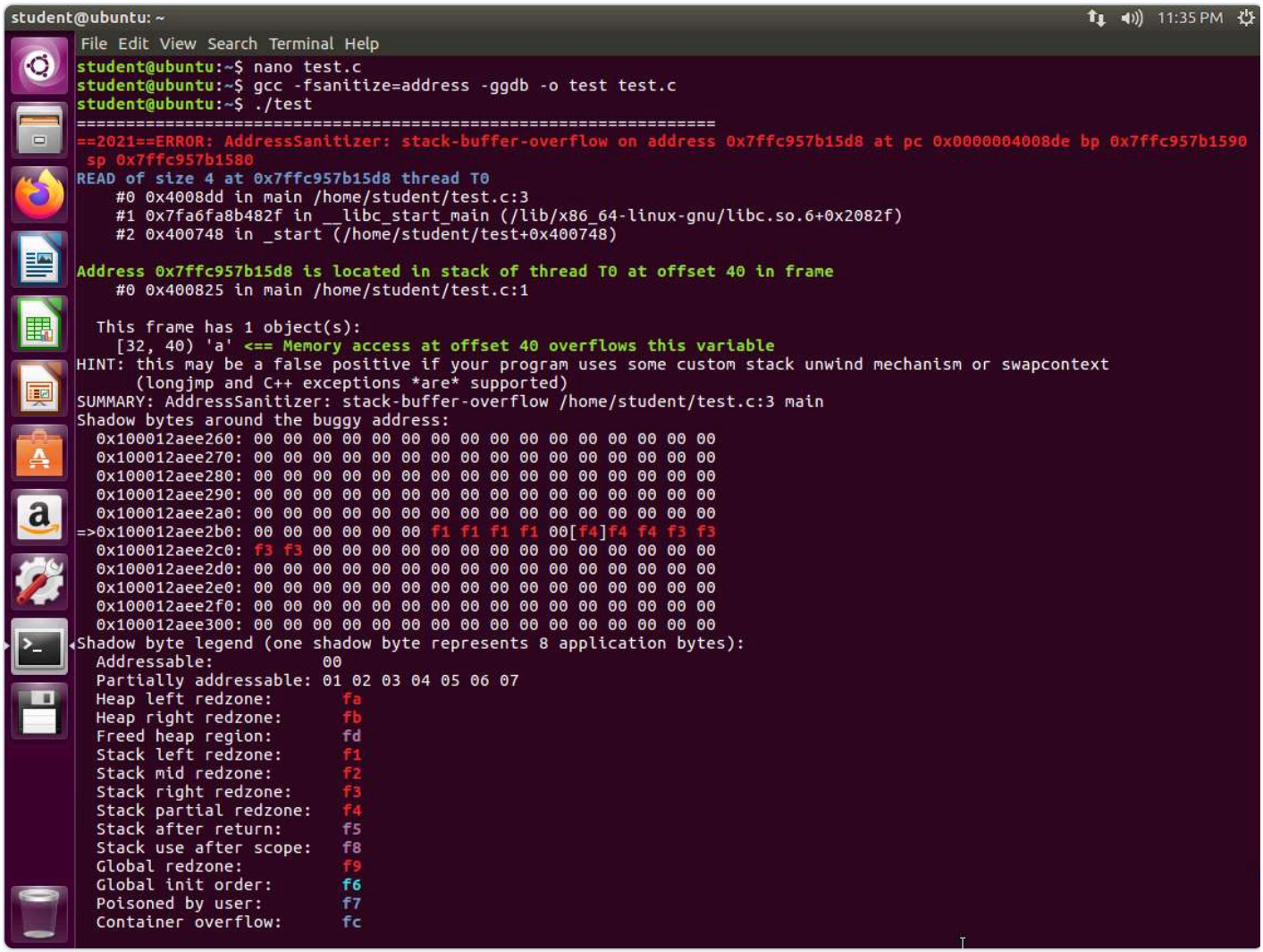
1. Log into **Ub20 Fuzzing** VM using credentials (**student** / **studentpassword**).

2. Create C source file **test.c** containing an out-of-bounds array access error:

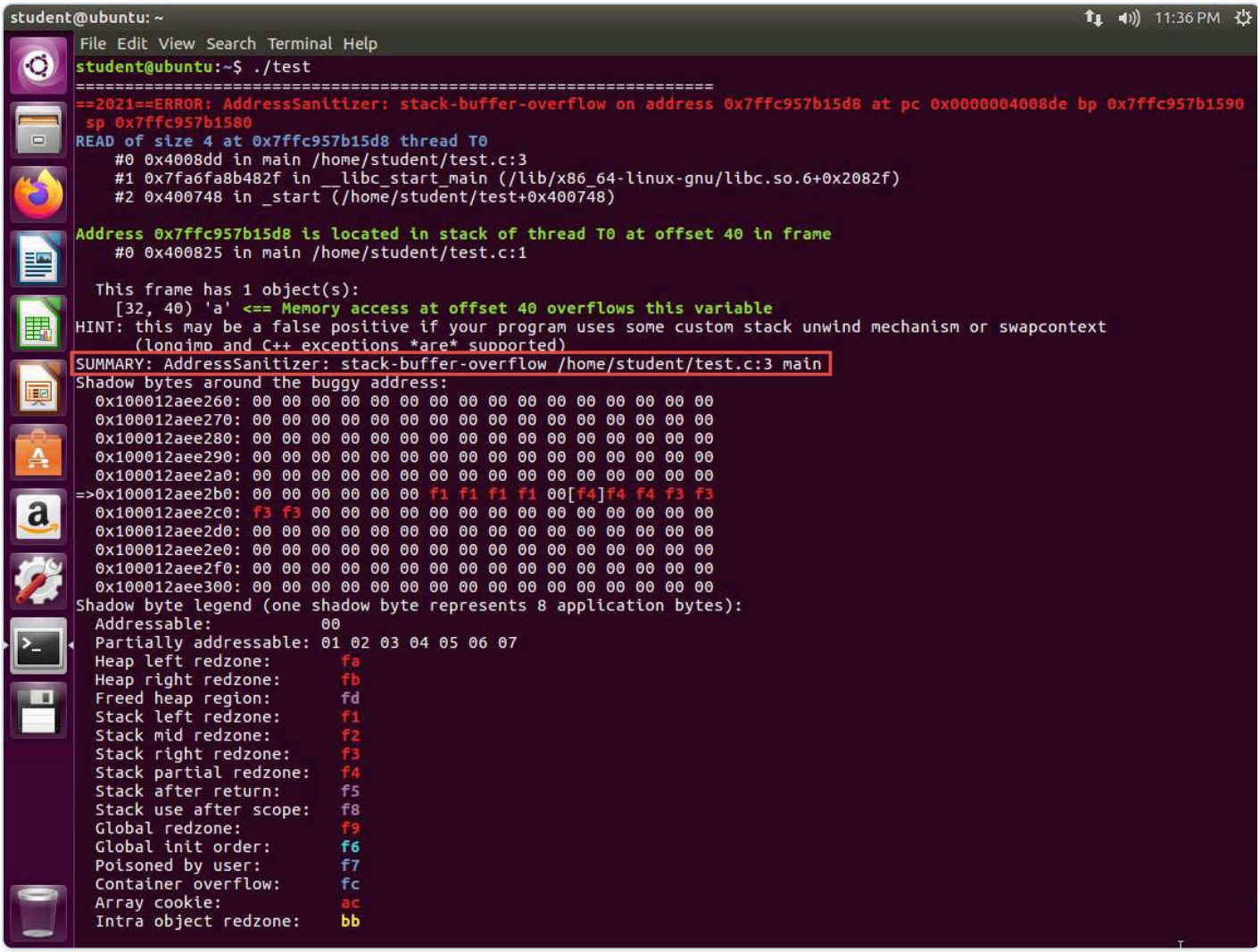
```
int main()   {
        int a[2] = {1,0};
        int b=a[2];
}
```

1. Compile source code using GCC with Address Sanitizer and GDB debug options:
   gcc -fsanitize=address -ggdb -o test test.c

2. Execute compiled binary:
   ./test



1. Inspect Address Sanitizer output identifying a `stack-buffer-overflow` at line 3 of `test.c`.



1. Configure build parameters for Autotools software packages:
   ./configure --disable-shared CFLAGS="-fsanitize=address -ggdb" CXXFLAGS="-fsanitize=address -ggdb" LDFLAGS="-fsanitize=address"
   make

2. Set environment variable to enforce application abort on memory violations during automated fuzzing:
   export ASAN_OPTIONS='abort_on_error=1'

This concludes the lab exercise.